

자료구조 실습 보고서

[제 11 주] 정렬 알고리즘의 비교

제출일 : 2018 년 5 월 22 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘 / 자료구조

- `public boolean sort(E[] aList, int aSize)` : 속해있는 클래스에 따라서 퀵 정렬 or 삽입 정렬
- 퀵 정렬 (오름차순)
 1. 정렬을 시작하기 이전에, 최댓값을 맨 오른쪽으로 옮겨놓은 후에 이를 제외한 리스트에 대하여 정렬을 시작한다.
 2. 기준이 되는 pivot 원소를 정한다. pivot 원소의 위치는 맨 왼쪽으로 한다.
 3. pivot 에 대하여, 맨 왼쪽과 맨 오른쪽에서부터 좁혀들어오며 탐색을 시작한다.
 4. 왼쪽에서 오른쪽으로 갈 때는 pivot 보다 큰 원소가 있을 때, 오른쪽에서 왼쪽으로 가는 경우는 pivot 보다 작은 원소가 있을 때 탐색을 멈춘다.
 5. 멈춘 인덱스의 원소끼리 교환한다.
 6. 좁혀들어오던 인덱스가 서로 교차되면 탐색을 멈추고, 왼쪽으로 가던 인덱스의 원소와 pivot 을 바꾼다.
 7. pivot 하나에 대하여 정렬이 완료된 리스트에서, pivot 양 쪽의 구간에 대하여 같은 작업을 반복한다.
- 삽입 정렬 (오름차순)
 1. 정렬을 시작하기 이전에, 최솟값을 맨 왼쪽으로 옮겨놓은 후에 이를 제외한 리스트에 대하여 정렬을 시작한다.
 2. 삽입할 원소(`insertedElement`) 를 두 번째 인덱스 부터 탐색하며, 동시에 그 이전 인덱스를 차례차례 내려가면서(`insertionLoc` 인덱스) 해당 위치의 원소가 삽입할 원소보다 클 때마다 둘 사이의 원소를 한 칸씩 뒤로 당긴다.
 3. `insertionLoc` 의 다음 위치에 `insertedElement` 를 넣는다.

b. 함수 설명서

■ AppController

- `public void run()` : 프로그램 실행

■ AppView

- `public void output(String aString)` : 문자열 출력
- `public void outputLine(String aString)` : 문자열 출력하고 줄바꿈

■ DataGenerator

- `public static Integer[] ascendingOrderList(int aSize)` : 오름차순 리스트 생성

- `public static Integer[] descendingOrderList(int aSize)` : 내림차순 리스트 생성
- `public static Integer[] randomOrderList(int aSize)` : 무작위 리스트 생성
- InsertionSort
 - `public boolean sort(E[] aList, int aSize)` : 주어진 크기의 리스트를 삽입정렬
- QuickSort
 - `public boolean sort(E[] aList, int aSize)` : 주어진 크기의 리스트를 퀵 정렬
- ListOrder
 - `public String orderName()` : ListOrder 형 필드에 할당된 리스트 유형에 맞는 한글 이름 반환

c. 종합 설명서

오름차순 리스트, 내림차순 리스트, 무작위 리스트를 각각 생성하여 삽입 정렬과 퀵 정렬의 두 가지 방법으로 정렬한 후에 결과를 검증한다.

2. 프로그램 장단점 분석

장점 : 정렬과 관련된 QuickSort, InsertionSort 클래스에서 중복되는 기능을 Sort 추상 클래스로부터 상속받았기 때문에, 쓸데없는 코드의 중복을 막았다.

단점 : x

3. 실행 결과 분석

a. 입력과 출력

```
<<< 정렬 알고리즘의 정렬 결과를 검증하는 프로그램을 시작합니다. >>>

> 정렬 결과의 검증 :

[오름차순 리스트] 의 앞 부분 : 0    1    2    3    4
[오름차순 리스트] 를 [InsertionSort] 한 결과는 올바릅니다.
[오름차순 리스트] 를 [QuickSort] 한 결과는 올바릅니다.

[내림차순 리스트] 의 앞 부분 : 9999  9998  9997  9996  9995
[내림차순 리스트] 를 [InsertionSort] 한 결과는 올바릅니다.
[내림차순 리스트] 를 [QuickSort] 한 결과는 올바릅니다.

[무작위 리스트] 의 앞 부분 : 4447  2404  6757  8959  7146
[무작위 리스트] 를 [InsertionSort] 한 결과는 올바릅니다.
[무작위 리스트] 를 [QuickSort] 한 결과는 올바릅니다.

<<< 정렬 알고리즘의 정렬 결과를 검증하는 프로그램을 종료합니다. >>>
```

b. 결과분석

정렬이 오름차순으로 잘 되었는지 검증하는 결과를 포함하였고, 여기서 정렬 결과가 올바르게 나왔으므로, 결과는 유효하다.

4. 소스코드

AppController.java

```
public class AppController {

    private static final int DEFAULT_TEST_SIZE = 10000;
    private static final int DEFAULT_FIRST_PART_SIZE = 5;
    private static final InsertionSort<Integer> INSERTION_SORT = new InsertionSort<Integer>();
    private static final QuickSort<Integer> QUICK_SORT = new QuickSort<Integer>();

    private AppView _appView;
    private Integer[] _list;
    private ListOrder _listOrder; // 리스트 종류 저장하는 임시 변수

    public AppController() {
        this._appView = new AppView();
    }

    public void run() {
        this._appView.outputLine("<<< 정렬 알고리즘의 정렬 결과를 검증하는 프로그램을 시작합니다. >>>");
        this._appView.outputLine("");

        this._appView.outputLine("> 정렬 결과의 검증 : ");
        this._appView.outputLine("");

        this.validateWithAscendingOrderList();
        this.validateWithDescendingOrderList();
        this.validateWithRandomOrderList();

        this._appView.outputLine("<<< 정렬 알고리즘의 정렬 결과를 검증하는 프로그램을 종료합니다. >>>");
    }

    // 비공개 함수

    // 오름차순 리스트 생성하여 보여준 후 정렬, 결과 검증
    private void validateWithAscendingOrderList() {
        this._listOrder = ListOrder.Ascending;
        this._list = DataGenerator.ascendingOrderList(DEFAULT_TEST_SIZE);
        this.showFirstPartOfDataList();
        this.validateSortsAndShowResult();
    }

    // 내림차순 리스트 생성하여 보여준 후 정렬, 결과 검증
    private void validateWithDescendingOrderList() {
        this._listOrder = ListOrder.Descending;
        this._list = DataGenerator.descendingOrderList(DEFAULT_TEST_SIZE);
        this.showFirstPartOfDataList();
        this.validateSortsAndShowResult();
    }
}
```

```

// 랜덤 리스트 생성하여 보여준 후 정렬, 결과 검증
private void validateWithRandomOrderList() {
    this._listOrder = ListOrder.Random;
    this._list = DataGenerator.randomOrderList(DEFAULT_TEST_SIZE);
    this.showFirstPartOfDataList();
    this.validateSortsAndShowResult();
}

// 정렬할 데이터 표본 출력
private void showFirstPartOfDataList() {
    this._appView.output(
        "[" + this._listOrder.orderName() + " 리스트] 의 앞 부분 : ");

    for(int i = 0; i < DEFAULT_FIRST_PART_SIZE; i++) {
        this._appView.output(" " + this._list[i].toString() + " ");
    }

    this._appView.outputLine("");
}

// 두 가지 방법으로 정렬한 후 검증하여 결과 출력 명령
private void validateSortsAndShowResult() {
    this.validateSort(AppController.INSERTION_SORT);
    this.validateSort(AppController.QUICK_SORT);
    this._appView.outputLine("");
}

// 매개 변수로 받은 방법으로 정렬 후 검증하여 결과 출력
private void validateSort(Sort<Integer> aSort) {
    Integer[] list = this.copyList(this._list); // 복사 : 원본 리스트는 유지되어야함 (여러 번 사용)
    aSort.sort(list, list.length);
    this.showValidationMessage(aSort, list);
}

// Integer[] 복사
private Integer[] copyList(Integer[] aList) {
    Integer[] copiedList = new Integer[aList.length];
    for(int i = 0; i < aList.length; i++) {
        copiedList[i] = aList[i];
    }
    return copiedList;
}

private void showValidationMessage(Sort<Integer> aSort, Integer[] aList) {
    this._appView.output(
        "[" + this._listOrder.orderName() + " 리스트] 를 [" +
        aSort.getClass().getSimpleName() + "] 한 결과는 ");
    if(this.sortedListIsValid(aList)) {
        this._appView.outputLine("올바릅니다.");
    }
    else {
        this._appView.outputLine("올바르지 않습니다.");
    }
}

// 오름차순인지 검증

```

```

        private boolean sortedListIsValid(Integer[] aList) {
            for(int i = 0; i < (aList.length - 1); i++) {
                if( aList[i].compareTo(aList[i + 1]) > 0 ) {
                    return false;
                }
            }
            return true;
        }
    }
}

```

AppView.java

// 출력 담당

```

public class AppView {

    public AppView() { }

    public void output(String aString) {
        System.out.print(aString);
    }

    public void outputLine(String aString) {
        System.out.println(aString);
    }

}

```

DataGenerator.java

// 데이터 생성

// static class를 구현하기 위하여, 모든 멤버는 static으로 선언하고 클래스는 final로

import java.util.Random;

```

public final class DataGenerator {

    // 인스턴스 생성 불가
    private DataGenerator() {}

    // 오름차순 리스트 생성
    public static Integer[] ascendingOrderList(int aSize) {
        Integer[] list = null;

        if(aSize > 0) {
            list = new Integer[aSize];
            for(int i = 0; i < aSize; i++) {
                list[i] = i;
            }
        }

        return list;
    }

    // 내림차순 리스트 생성
    public static Integer[] descendingOrderList(int aSize) {
        Integer[] list = null;
    }
}

```

```

        if(aSize > 0) {
            list = new Integer[aSize];
            for(int i = 0; i < aSize; i++) {
                list[i] = aSize - 1 - i;
            }
        }

        return list;
    }

}

// 무작위 리스트 생성
public static Integer[] randomOrderList(int aSize) {
    Integer[] list = null;

    if(aSize > 0) {
        list = new Integer[aSize];
        for(int i = 0; i < aSize; i++) {
            list[i] = i;
        }

        Random random = new Random();
        for(int i = 0; i < aSize; i++) {
            int j = random.nextInt(aSize);
            Integer temp = list[i];
            list[i] = list[j];
            list[j] = temp;
        }
    }

    return list;
}

}

```

ListOrder.java

```

// 리스트의 종류 구분

public enum ListOrder {

    Ascending,           // 오름차순 리스트 유형
    Descending,          // 내림차순 리스트 유형
    Random;              // 무작위 리스트 유형

    public static final String[] ORDER_NAMES = {"오름차순", "내림차순", "무작위"};

    // ListOrder 인스턴스가 가진 리스트 유형의 한글명 반환
    public String orderName() {
        return ListOrder.ORDER_NAMES[this.ordinal()];
    }

}

```

Sort.java

```

// 정렬의 기본 베이스
// 추상 클래스 : 인스턴스 생성할 때 반드시 상속이 필요

public abstract class Sort<E extends Comparable<E>> {

    protected void swap(E[] aList, int i, int j) {
        E tempElement = aList[i];
    }
}

```

```

        aList[i] = aList[j];
        aList[j] = tempElement;
    }

    // 생성자
    protected Sort() {}

    // 정렬
    public abstract boolean sort(E[] aList, int aSize);
}

```

InsertionSort.java

```

    // 삽입 정렬

    public class InsertionSort<E extends Comparable<E>> extends Sort<E> {

        // 생성자
        public InsertionSort() {}

        // 삽입 정렬
        public boolean sort(E[] aList, int aSize) {

            if( (aSize < 1) || (aSize > aList.length) ) {
                return false;
            }

            // 최솟값 위치 탐색 후 0 번 인덱스로 이동
            int minLoc = 0;
            for(int i = 1; i < aSize; i++) {
                if( aList[i].compareTo(aList[minLoc]) < 0 ) {
                    minLoc = i;
                }
            }
            this.swap(aList, 0, minLoc);

            // 리스트 탐색해가며 맞는 위치에 삽입
            for(int i = 2; i < aSize; i++) {
                E insertedElement = aList[i]; // 대상 원소를 미리 저장해놓음 (원래 위치에서 덮어쓰워짐)
                int insertionLoc = i - 1;

                // 삽입할 원소보다 더 크다면 거기서부터 원소 한칸씩 뒤로 당김 (앞쪽의 공간 생성)
                while(aList[insertionLoc].compareTo(insertedElement) > 0) {
                    aList[insertionLoc + 1] = aList[insertionLoc];
                    insertionLoc--;
                }

                // 삽입할 원소보다 더 작은 원소를 가리키게 됨
                // 최솟값은 0번 인덱스에 존재하기 때문에 끝까지 가도 벗어나
                aList[insertionLoc + 1] = insertedElement;
            }

            return true;
        }
    }
}

```



```
public class QuickSort<E extends Comparable<E>> extends Sort<E> {

    // 생성자
    public QuickSort() {}

    public boolean sort(E[] aList, int aSize) {
        if(aSize < 1 || aSize > aList.length) {
            return false;
        }
        else {

            // 최댓값 위치 탐색 후 맨 끝 인덱스로 이동
            int maxLoc = 0;
            for(int i = 0; i < aSize - 1; i++) {
                if( aList[i].compareTo(aList[maxLoc]) > 0 ) {
                    maxLoc = i;
                }
            }

            if( aList[maxLoc].compareTo(aList[aSize - 1]) > 0 ) {
                this.swap(aList, aSize - 1, maxLoc);
            }
            this.quickSortRecursively(aList, 0, aSize - 2);
            return true;
        }
    }

    // 비공개 함수

    // pivot 위치 결정 : 왼쪽으로
    private int pivot(E[] aList, int left, int right) {
        return left;
    }

    // partition 수행 후 pivot 위치 반환
    private int partition(E[] aList, int left, int right) {

        int pivot = this.pivot(aList, left, right);
        int toRightIndex = left + 1;
        int toLeftIndex = right;

        while(toRightIndex <= toLeftIndex) {

            // toRightIndex : pivot 보다 작을 시 맞는 위치 (오름차순)
            while( aList[toRightIndex].compareTo(aList[pivot]) < 0 ) {
                toRightIndex++;
            }

            // toLeftIndex : pivot 보다 클 시 맞는 위치 (오름차순)
            while( aList[toLeftIndex].compareTo(aList[pivot]) > 0 ) {
                toLeftIndex--;
            }

            if(toRightIndex < toLeftIndex) {
                this.swap(aList, toRightIndex, toLeftIndex);
            }
        }
    }
}
```

```

        this.swap(aList, toLeftIndex, pivot);

        // pivot 위치 반환
        return toLeftIndex;
    }

    // 재귀정렬
    private void quickSortRecursively(E[] aList, int left, int right) {
        if(left < right) {
            int mid = this.partition(aList, left, right);
            this.quickSortRecursively(aList, left, mid - 1);
            this.quickSortRecursively(aList, mid + 1, right);
        }
        // 정렬이 끝난 상태라면 아무것도 하지 않음
    }
}

```

DS1_11_201702081_최재범.java

```

public class DS1_11_201702081_최재범 {

    public static void main(String[] args) {

        AppController appController = new AppController();
        appController.run();

    }
}

```